

Визуальное моделирование при анализе и проектировании. Основы Unified Modeling Language (UML)

Ищенко Алексей Петрович

Анализ и проектирование

Решение задач с использованием вычислительной техники. При этом, особое внимание необходимо уделить тому, что непосредственное программирование или написание кода начинается далеко не сразу. Более того, этапы, предшествующие разработке не менее важны и сложны.

Примерная схема может выглядеть так:



Анализ – Проектирование - Программирование

- В настоящий момент в анализе и проектировании преобладает **объектный подход**

2 вида разбиения предметной области на составляющие
(**Декомпозиция**):

- **Алгоритмическая декомпозиция**

Основные элементы – **алгоритмы**.

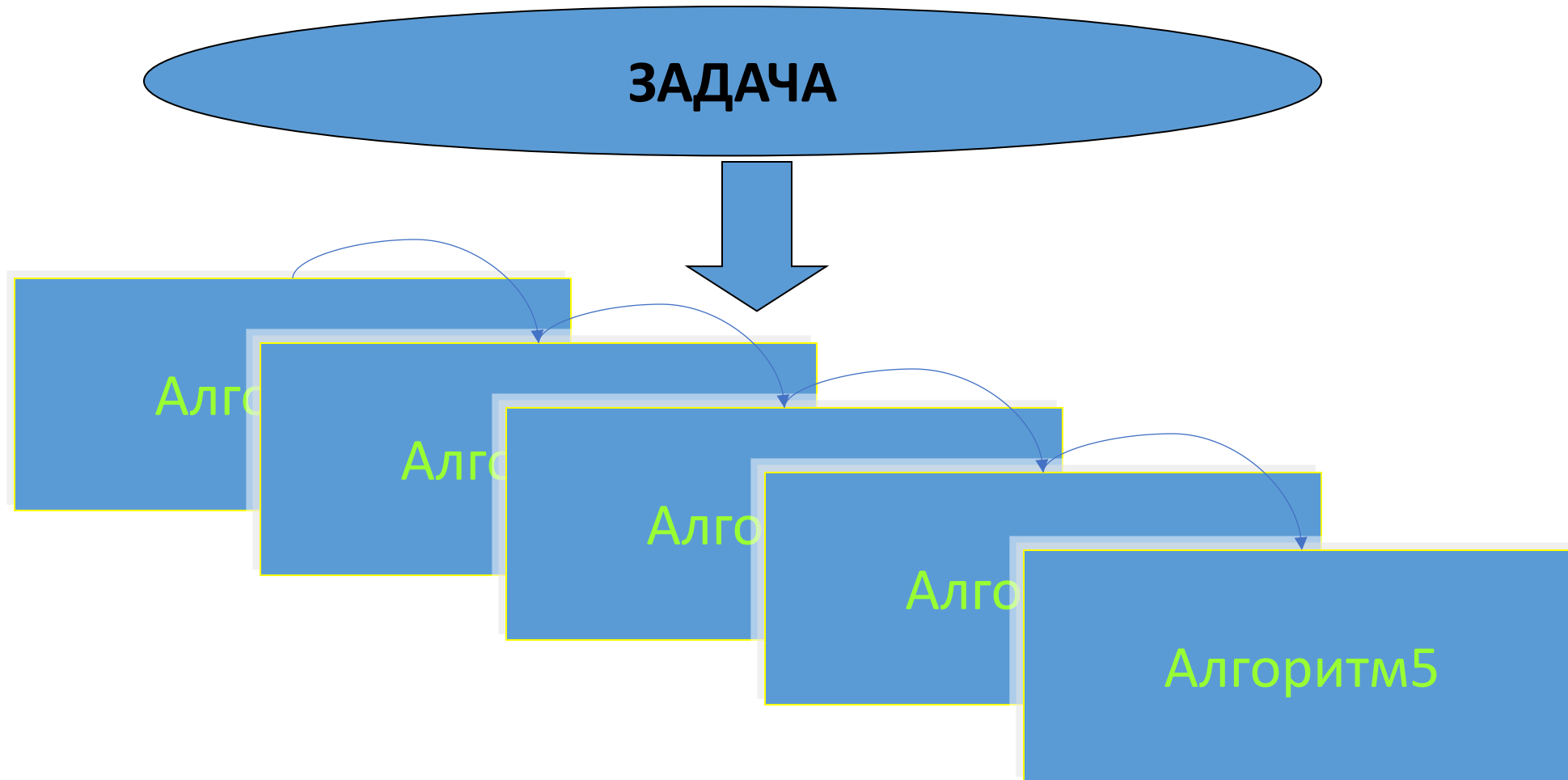
- **Объектная декомпозиция**

Основные элементы – виды абстракций (**классы**) и представители этих классов (**объекты**)

Алгоритмическая декомпозиция

- В соответствии с алгоритмической декомпозицией предметной области мы при анализе задачи пытаемся понять, какие алгоритмы необходимо разработать для ее решения, каковы спецификации этих алгоритмов (вход, выход), и как эти алгоритмы связаны друг с другом. В языках программирования данный подход в полной мере поддерживается средствами модульного программирования (библиотеки, модули, подпрограммы).

Алгоритмическая декомпозиция



Объектный подход

- В рамках объектной декомпозиции мы пытаемся выделить основные содержательные элементы задачи, разбить их на типы (классы). Далее для каждого класса абстракций мы определяем его свойства (данные) и поведение (операции), а также, как эти классы абстракций взаимодействуют друг с другом.
- На сегодняшний день объектный подход и его основы – объектная модель и объектная декомпозиция – поддерживаются современными объектно-ориентированными языками программирования (Object Pascal, C++, Java, C#...).

Объектный подход

Объектный подход:

- **OOA** (object oriented analysis) – *объектно-ориентированный анализ.*
- **OOD** (object oriented design) – *объектно-ориентированное проектирование.*
- **OOP** (object oriented programming) – *объектно-ориентированное программирование.*

Принципы объектного подхода

- **Абстрагирование.**

выделяем главное, выявляем виды абстракций

- **Инкапсуляция.**

скрываем детали реализации

- **Иерархия.**

иерархия помогает разбить задачу на уровни и постепенно ее решать

- **Агрегация и наследование.**

абстракции можно создавать на основе имеющихся

- **Полиморфизм.**

полиморфизм позволяет иметь естественные имена и выполнять действия, релевантные ситуации, разбираясь на этапе работы программы

Повторное использование...

- **Повторное использование** – применение уже существующих наработок в разрабатываемом ПО.
- Повторное использование – важный элемент проектирования.
 - Необходимо проектировать новые элементы системы с тем, чтобы их в последствии можно было использовать.
 - Необходимо при проектировании системы рассматривать возможность использования того, что уже есть и работает.
- Достоинства повторного использования (по Соммервиллю):
 - Повышение надежности.
 - Уменьшение проектных рисков.
 - Эффективное использование специалистов.
 - Соблюдение стандартов (пример: UI).
 - Ускорение разработки.

Повторное использование. Виды

Повторное использование достигается за счет следующих приемов:

- **Компонентная разработка.**

Часть компонентов уже разработаны ранее, имеют четко описанный интерфейс. Они используются в качестве «кирпичиков» в новой системе.

- **Использование паттернов (шаблонов) проектирования.**

Применяются известные подходы к решению некоторых встречавшихся ранее проблем.

- **Использование стандартных прикладных (MKL, MFC...) и системных (API) библиотек.**

UML

При изучении материалов по визуальному моделированию и языку UML в качестве основного источника рекомендуется классическая книга:

Г. Буч, Дж. Рамбо, А. Джекобсон. UML. Руководство пользователя. — ДМК-Пресс, Питер, 2004.

Модель

- Проблема в разработке ПО:

Проекты не укладываются в сроки, бюджет, не удовлетворяют требованиям.

Как бороться? На помощь приходит **моделирование**.

- **Модель** – упрощенное представление объектов и явлений реального мира.

Смысл моделирования

- **Модель** строят для того, чтобы лучше понять исследуемую систему.
- **Задачи моделирования:**
 - Визуализация системы в ее некотором состоянии.
 - Определение структуры и поведения системы.
 - Получение шаблона для создания системы.
 - Документирование принятых решений.

Принципы моделирования

- **Выбор модели** оказывает определяющее влияние на подход к решению проблемы и на то, как будет выглядеть это решение.
- Каждая модель может быть воплощена с разной **степенью абстракции**.
- Лучшие модели – те, что **ближе к реальности**.
- Наилучший подход при разработке сложной системы – использовать **несколько** почти независимых **моделей**.

Моделирование и объектный подход

- **Объектный подход** — один из ключевых подходов к моделированию.

В результате OOA & OOD мы получаем «хороший» проект программной системы, прозрачный, удовлетворяющий требованиям, удобный для тестирования и отладки, коллективной разработки, развиваемый, допускающий повторное использование компонентов.

- **В больших системах проект слишком велик для восприятия одним человеком.**

Идея визуального моделирования

- Путь к решению проблемы:
ВИЗУАЛЬНОЕ МОДЕЛИРОВАНИЕ
- В чем смысл?
 - **Визуализация** упрощает понимание проекта в целом.
 - **Визуализация** помогает согласовать терминологию и убедиться, что все одинаково понимают термины.
 - **Визуализация** делает обсуждение конструктивным и понятным.

UML как воплощение идеи визуального моделирования

- Для визуального моделирования нужна специальная нотация или **язык**.
- **UML** (unified modeling language) – это **язык** для
 - визуализации,
 - специфицирования,
 - конструирования,
 - документированияэлементов программных систем [3].
- **UML** – язык общего назначения, предназначенный для объектного моделирования.

Модели UML

UML позволяет описывать систему следующими **моделями**:

- **Модель функционирования**

Как описывается функциональность системы с точки зрения пользователя.

- **Объектная модель**

Как выглядит проект системы с точки зрения объектного подхода.

- **Динамическая модель**

Как взаимодействуют друг с другом компоненты системы в динамике, с течением времени. Какие процессы происходят в системе.

Диаграммы UML

- **Диаграммы UML** предназначены для визуального отображения **моделей** и их компонентов.
- **UML 2.0 – 13** типов диаграмм.
 - Структурные диаграммы (6)
 - Диаграммы поведения (3)
 - Диаграммы взаимодействия (4)

Структурные диаграммы

- **Диаграмма классов**

Показывает классы, их атрибуты и связи между классами.

- **Диаграмма компонентов**

Показывает компоненты и связи между ними

- **Структурная диаграмма**

Показывает внутреннюю структуру классов и связи с внешним миром

- **Диаграмма развертывания**

Показывает, как ПО размещается на аппаратуре (серверах, рабочих станциях...)

- **Диаграмма объектов**

Показывает структуру системы в конкретный момент времени, объекты, их атрибуты...

- **Диаграмма пакетов**

Показывает, как система раскладывается на крупные составные части и связи между этими частями

Диаграммы поведения

- **Диаграмма действия**

Показывает потоки информации в системе.

- **Диаграмма состояния**

Представляет собой конечный автомат, показывающий функционирование системы.

- **Диаграмма вариантов использования**

Показывает работу системы с точки зрения пользователей.

Диаграммы взаимодействия

- **Диаграмма кооперации**

Показывает структурную организацию участвующих во взаимодействии объектов

- **Диаграмма взаимодействия**
(новация UML 2.0)

- **Диаграмма последовательности**

Показывает временную упорядоченность событий

- **Временная диаграмма**

Диаграмма связана с временными рамками

Понятия UML

- **Для описания структуры:**

Актер, Атрибут, Класс, Компонент, Интерфейс, Объект, Пакет.

- **Для описания поведения:**

Действие, Событие, Сообщение, Метод, Операция, Состояние, Вариант использования.

- **Для описания связей:**

Агрегация, Ассоциация, Композиция, Зависимость, Наследование.

- **Некоторые другие понятия:**

Стереотип, Кратность, Роль.

Пример. Система бронирования билетов для авиакомпании

- **SRS** – Seat reservation system.
- Авиакомпания «GlobalAvia».
- SRS должна содержать 2 части:
 - Занесение информации.
 - Работа с клиентами.
- Дополнительная информация:
 - Рейсы спланированы так, что до пункта назначения можно долететь с пересадками.
 - Система должна помогать покупать билеты в зависимости от пожеланий пользователя.

Как функционирует программная система?

- Программная система **не функционирует сама по себе.**
- Программная система функционирует под воздействием **актеров – пользователей, машин и других программ.**
- **Актер** ожидает, что система ведет себя строго определенным образом.
- **Актер** оказывает **воздействие** – система выдает ожидаемый **результат.**
- Модель того, как воздействие приводит к результату – **Вариант использования.**

Актеры и Варианты использования в UML

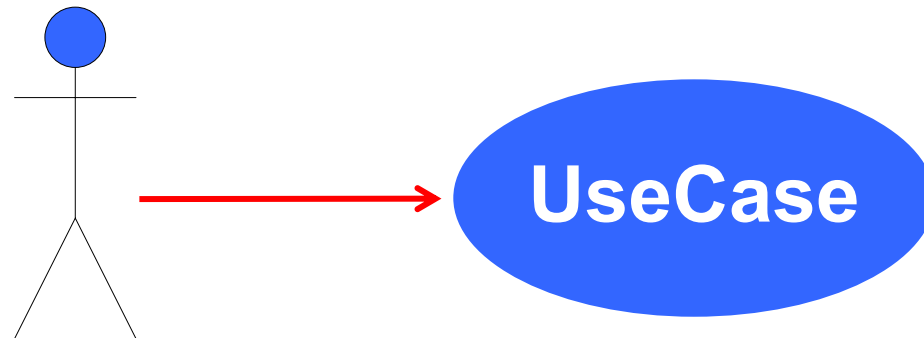
Актер в UML – человек, машина или программа, воздействует на систему, является внешним по отношению к ней.



Вариант использования в UML – описание последовательности действий – (часто с вариантами – сценариями).

Связь актеров и вариантов использования

- Актеры и варианты использования общаются посредством посылки сообщений.
- Сообщения могут идти в обе стороны.
- Стрелка показывает инициатора общения (актер на рисунке) и может быть опущена.



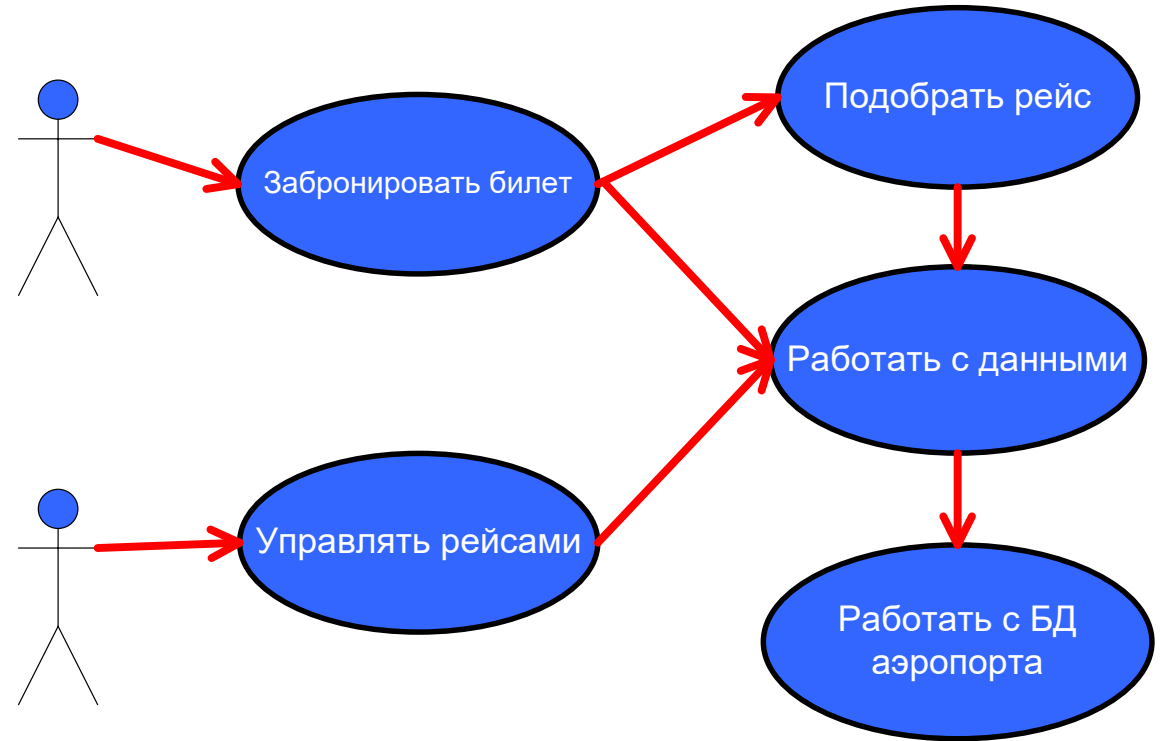
Актеры и Варианты использования в SRS

- **Актеры:**

- Пользователь.
- Администратор.

- **Варианты использования:**

- Забронировать билет.
- Подобрать рейс.
- Работать с данными.
- Управлять рейсами.
- Работать с БД аэропорта.



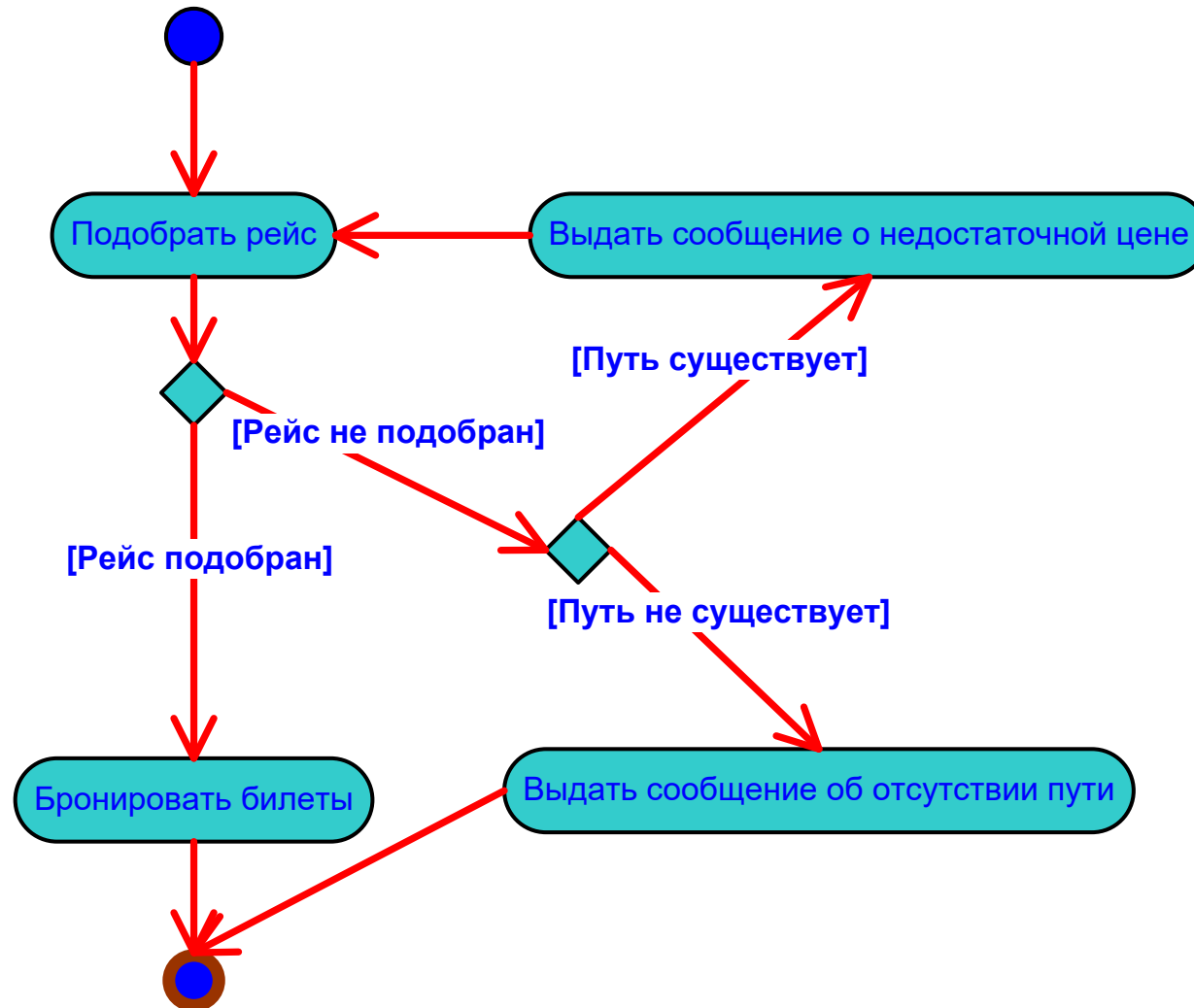
Некоторые соображения

- При таком моделировании обращают внимание на **поведение** системы, а не на ее реализацию.
- Хорошая модель описывает **основное поведение** системы, не являясь слишком подробным.
- Подобная модель позволяет проверить, удовлетворит ли система требования заказчика.
- Система средних размеров может быть описана большим количеством вариантов использования.
- Варианты использования могут описываться разными **сценариями**.

Сценарии варианта использования

- Для описания **сценариев Варианта использования** используется **Диаграмма действия**.
- Диаграмма действия это блок-схема, которая отображает динамику в поведении системы.
- Может использоваться **не только** для описания сценариев Варианта использования.

Диаграммы действия в SRS



Спасибо за внимание!

Далее, (вторая часть)

Структура системы и ее описание средствами UML:

- Классы (шаблоны классов)
- Объекты
- Интерфейсы
- Пакеты
- Подсистемы
- Компоненты
- Комментарии